Maynooth University
National University of Ireland Maynooth

MSc. Data Science and Analytics Thesis

truncpois

An R Package for Truncated Poisson Distributions

Author: Arun Sundar Paulraj

Student Number: 25250740

Supervisor: Dr. Keefe Murphy

A thesis submitted in fulfilment of the requirements for the degree of Masters in Data Science and Analytics 2025-2026

to the

Department of Mathematics & Statistics

Maynooth University

1st August 2026

Acknowledgements

I would like to sincerely thank my supervisor, Dr. Keefe Murphy, for his guidance, support, and valuable feedback throughout the development of the `truncpois` package and the writing of this thesis.

Abstract

Although there are a variety of applications of count-data for which some types of observations are either not made or otherwise cannot be observed (e.g., zero-truncated counts in studies of species abundance, insurance claims, reported cases of disease), available R implementations for the truncated Poisson, with the exception of the `LaplacesDemon` package, do not fully support this type of model; that is to say, they have no way to calculate moments using analytical means for an arbitrary number of parameters, do not completely address how to compute probabilities in the right tail and/or at the log scale, and rely on Monte Carlo simulation to estimate moments that can instead be computed exactly.

This dissertation provides `truncpois`, an R package that implements all aspects of the (left-, right- and doubly-) truncated Poisson distribution in a numerically stable form. It includes all standard density, distribution, quantile, and random sampling functions. Closed-form expressions for the mean and variance are derived using the Poisson recurrence relation, while the median and mode are obtained through complementary exact, deterministic methods. The entire computation is carried out on the log-scale to avoid overflow when dealing with large parameter values; furthermore, three alternative random sampling algorithms have been developed for use with different truncations. In addition to these functionalities, it also develops a maximum likelihood estimator to obtain the rate parameter from observed counts as well as a visualization tool to compare the distributions of the truncated and untruncated cases.

Six experiments demonstrate the reliability, accuracy, and computational robustness of this package. These experiments include visualizing the probability mass function of the distribution; demonstrating how the truncated moments converge to their non-truncated counterparts; showing how the sampling methods perform relative to each other; evaluating the computational stability as the parameters take on extreme values while at the same time the truncation window is very narrow; performing a simulation study using the zero-truncated Poisson distribution; and recovering estimated parameters via maximum likelihood estimation. Overall, this demonstrates that `truncpois` fills a significant void in R's available options for analyzing truncated counts with exact, replicable and computationally efficient algorithms, whereas previous approximations were incomplete.

Keywords: truncated Poisson distribution, R package, numerical stability, maximum likelihood estimation, closed-form moments.

Contents

List of tables

List of figures

1 Introduction

Truncated probability distributions are a common area of study within statistics, as they occur frequently in practice when the range of valid observations has been limited in some way. The truncated Poisson distribution is an important type of model that can be used to fit count-type data, such as in studies concerning species abundance, insurance claims and disease prevalence surveys (where the number of cases is never equal to zero).

Truncated distributions have been studied extensively in the theoretical literature, notably by Nadarajah and Kotz (2006), who provide both a general theoretical treatment and accompanying R code for computing truncated versions of a wide range of distributions. Despite this, existing software implementations of the truncated Poisson distribution, including `LaplacesDemon` and the general-purpose `truncdist` package (itself associated with Nadarajah and Kotz (2006)), remain computationally inefficient and numerically unstable with respect to extreme parameter values. In particular, these implementations typically do not support computation on the log scale, nor do they provide the upper-tail and log-probability arguments needed to obtain such quantities without a manual, and potentially numerically unstable, adjustment; they rely on numerical integration or Monte Carlo simulation to obtain the mean and variance even though closed-form solutions exist for the Poisson distribution; and, since they are not designed specifically for discrete distributions, they do not correctly adjust the lower truncation bound to account for the non-zero probability mass located there.

The `truncpois` R package addresses these gaps in a number of ways.

Firstly, full distribution functions are provided in accordance with the d/p/q/r convention of `R`, including the probability mass function (PMF), via `dtruncpois`, cumulative distribution function (CDF), via `ptruncpois`, quantile function, via `qtruncpois`, and random number generation, via `rtruncpois`.

Secondly, where relevant, these functions allow all combinations of the logical arguments `log`, `log.p`, and `lower.tail`. Relatedly, all computations are done on the log scale to avoid problems of numerical underflow and/or overflow when parameters and/or truncation limits are at extremes. In addition, these functions properly accommodate the inclusive truncation bounds, which is inadequately addressed for discrete distributions by the code of Nadarajah and Kotz (2006).

Thirdly, exact moment computations are provided for the mean and variance, using closed-form formulas based on the Poisson recurrence relation. Notably, the code of Nadarajah and Kotz (2006) relies on numerical integration for its moment calculations, which is entirely inappropriate for discrete distributions such as the truncated Poisson.

Fourthly, we provide additional novel functionalities that were not available in the code of Nadarajah and Kotz (2006) to calculate the median and mode of truncated Poisson distributions, again in an exact and deterministic way. Other new contributions include the provision of a plotting function, for visualising the PMF, CDF, and quantile function of a truncated Poisson distribution and overlaying a comparison against the corresponding non-truncated Poisson distribution.

Fifthly, we provide three different sampling algorithms for `rtruncpois`, only one of which was provided by Nadarajah and Kotz (2006). We demonstrate the advantages of our novel schemes through some numerical experiments which follow in Section 4.

Finally, we note that the special cases of left-truncation, right-truncation, double-truncation and no truncation are accommodated throughout all functions in this package, with suitable defaults specified where relevant to correspond to the standard, non-truncated Poisson distribution.

The remainder of this thesis is structured as follows. In Section 2, we review relevant theoretical background for the Poisson distribution and univariate discrete distributions and survey existing software implementations. In Section 3, we outline specific details of the `truncpois` software implementation, describing the identities on which our various functions rely. In Section 4, we conduct a number of experiments which showcase the advantages of `truncpois` over existing software implementations. Finally, we conclude in Section 5 with a discussion.

2 Background and Literature Review

In this Section, we review the theoretical background necessary to understand truncated discrete distributions and the truncated Poisson distribution in particular, before surveying existing software implementations and identifying the specific shortcomings that `truncpois` is designed to address. Specifically, Section 2.1 introduces truncated distributions, the truncation regimes considered throughout this thesis, and the truncated Poisson probability mass function, while Section 2.2 reviews existing R packages that provide some support for truncated distributions and discusses their limitations relative to `truncpois`.

2.1 Truncated Distributions

When data generated by a random variable X cannot take on any value other than those included in a limited set of possible values (because such values may be structurally impossible, or merely omitted), this results in a truncated distribution. A truncated distribution differs from censored data, which includes some information about out-of-range values but does not include actual values for them; in the case of censoring there is an indication that a value did exist, whereas with truncation, no data is collected that would indicate that such a value exists. The total probability or density associated with a value not in the support has been redistributed among the remaining values, along with appropriate adjustments made to the normalisation factor.

Three truncation regimes are commonly distinguished for a discrete random variable with support constrained to $[a, b]$:

- **Left-truncation** ($a > 0$, $b = \infty$): the most common case in count-data applications, where counts below some threshold cannot be observed. The special case $a = 1$ is known as *zero-truncation*, and arises whenever the very fact of being included in a sample requires at least one occurrence of the

event being counted, for example, a respondent is only included in a survey of hospital visits if they visited hospital at least once.

- **Right-truncation** ($a = 0$, $b < \infty$): counts above some upper threshold are excluded, for instance when a data-collection instrument imposes a maximum recordable value.
- **Doubly-truncated** ($0 < a \leq b < \infty$): both bounds are finite, restricting observations to a bounded window.

For a truncated Poisson distribution with support $[a, b]$, the probability mass function is

$$P(X = k \mid a \leq X \leq b) = \frac{P(X = k \mid \lambda)}{P(a \leq X \leq b)}, \qquad a \leq k \leq b,$$

where $P(X = k \mid \lambda)$ denotes the probability mass function of the corresponding non-truncated Poisson distribution. This is parameterised by a strictly positive rate parameter λ, and its probability mass function is given by

$$P(X = k \mid \lambda) = \frac{\lambda^k e^{-\lambda}}{k!}, \qquad k = 0, 1, 2, \ldots,$$

such that the non-truncated distribution corresponds to the special case $a = 0$, $b = \infty$. The denominator $P(a \leq X \leq b)$ is the normalising constant, and can equivalently be written in terms of the non-truncated cumulative distribution function as $P(a \leq X \leq b) = P(X \leq b) - P(X \leq a)$.

Zero-truncation is sometimes confused with *zero-inflation* in applied statistical modelling, but the two are entirely different concepts. Zero-truncation refers to the complete removal of zero from the sample space, so that a zero count is structurally impossible to observe, whereas zero-inflation instead adds an excess point mass at zero, over and above what a standard Poisson distribution would predict, to accommodate data with more zeroes than the model would otherwise allow. The `truncpois` package addresses only the former; zero-inflated count models are a distinct modelling problem, typically fitted with dedicated packages, and are not addressed here.

Data collected on count variables that experience truncation like this is quite common in many different types of applications. For example, abundance surveys of species in ecology often do not include all species present in the area being surveyed because those with a zero count have no record of their presence; likewise, insurance claim data typically will not contain records of claims that did not occur (since zero-claim policies will never generate an actual claim); finally, case counts for diseases are only reported once a threshold number of cases has been reached, so counts below that threshold are never observed. As a result, fitting an unmodified Poisson model to data from any of these areas, without accounting for the truncated region of the Poisson distribution's support, will produce biased parameter estimates.

2.2 Existing Software Implementations

Several R packages offer some support for truncated distributions, but none combines Poisson-specific closed-form moments with fully general double truncation and complete log-scale support. Five are considered here.

LaplacesDemon. The `LaplacesDemon` package (Statisticat, LLC 2026) provides a truncated Poisson distribution, with `dtrunc()`, `ptrunc()`, `qtrunc()`, and `rtrunc()` functions (via `spec = "pois"`) for density, distribution, quantile, and random-generation calculations respectively. Its PMF and CDF values agree with `truncpois`, but the package has several notable limitations that motivated design choices in `truncpois`. First, `ptrunc()` and `qtrunc()` always assume `lower.tail = TRUE`, `log = FALSE`, and `log.p = FALSE`; they do not support the `lower.tail` and `log.p` arguments that `truncpois` provides natively. Obtaining an upper-tail or log-scale probability therefore requires a manual `1 - p` (or `log(1 - p)`) transformation. This is not merely imprecise or risky, but actually incorrect in general: `LaplacesDemon`'s truncated distribution functions are built from several internal calls to the corresponding non-truncated Poisson functions (analogous to g and G, introduced in Section 3), and applying a `1 - p` or `log(1 - p)` adjustment only to the final output does not correctly propagate through these intermediate calls; the resulting value can therefore differ from the true upper-tail or log-scale probability, rather than merely being numerically imprecise. Second, `LaplacesDemon` provides no closed-form mean or variance for the truncated Poisson distribution; obtaining these requires Monte Carlo simulation via `rtrunc()`, which is slower and introduces sampling variability that grows worse for extreme truncation parameters, when numerical integration (itself invalid for a discrete distribution, and unnecessary given the closed-form Poisson recurrence relation used by `truncpois`) or exact summation over the support would be the only genuinely appropriate alternatives. The median and mode are not provided by `LaplacesDemon` at all, whether via simulation, integration, or any other means.

truncdist. Nadarajah and Kotz (2006) also published an associated R package, `truncdist` (Novomestky and Nadarajah 2016), which provides generalised (distribution-agnostic) `dtrunc()`, `ptrunc()`, `qtrunc()`, `rtrunc()` wrapper functions for truncating any distribution supplied by `stats`, `stats4`, or `evd`. While this generality is useful for some purposes, it makes `truncdist` unsuitable for the case at hand, since there is no way to exploit the specific closed-form recurrence relation of the Poisson distribution that `truncpois` relies on to compute exact moments, nor does `truncdist` provide a log-scale computational path. As a package built on the same general-purpose principles as `LaplacesDemon`, `truncdist` inherits the same limitations described above: it assumes `lower.tail = TRUE`, `log = FALSE`, and `log.p = FALSE` throughout, and offers no closed-form median or mode.

VGAM and VGAMdata. The `pospoisson()` family in the `VGAM` package (Yee 2025a) allows a *zero-truncated (or positive) Poisson* regression to be fitted using `vglm()`. The companion `VGAMdata` package (Yee 2025b) provides the corresponding standalone distribution functions, `dpospois()`, `ppospois()`, `qpospois()`, and `rpospois()`, but these are restricted to the zero-truncated case; they do not allow right-truncation or double-truncation, and inherit the same `lower.tail`/`log.p`/`log` limitations described above.

gamlss.tr and countreg. The two remaining packages use truncation to construct new distributions for use in regression modelling, rather than as stand-alone utilities for a specific truncated distribution. `gamlss.tr` (Stasinopoulos and Rigby 2024) uses `gen.trun()` to create a truncated version of any `gamlss.family` distribution for use in a GAMLSS regression model. `countreg` (Zeileis and Kleiber 2024) (currently available on R-Forge) allows zero-truncated count regression models to be fitted via `zerotrunc()` or `ztpoisson`, either of which may be used as the count component of a hurdle model. Neither package provides the exact closed-form moments of a stand-alone (non-regression) truncated Poisson distribution that are the focus of this thesis, and both inherit the log-scale and tail-probability limitations described above.

A limitation common to all five of the packages just discussed, beyond the `log`/`log.p`/`lower.tail` issues and the reliance on numerical integration or Monte Carlo simulation for moments, is that none of them adjust the lower truncation bound to account for the discrete nature of the distribution. For a continuous distribution, $P(X = a) = 0$ by definition, so no such adjustment is needed; however, for a discrete distribution such as the Poisson, $P(X = a) \geq 0$, and computing $P(a \leq X \leq b)$ correctly therefore requires evaluating the non-truncated CDF at $a-1$ rather than at a, so that the probability mass located exactly at the lower truncation bound is correctly included rather than excluded. `truncpois` performs this $a \to a-1$ adjustment throughout; none of the packages reviewed above do so, which introduces a further, unacknowledged source of error whenever they are applied to the left-truncated or doubly-truncated case.

3 Package Implementation

Throughout this Section, we describe each of the ten exported functions provided by `truncpois`, focusing on the mathematical identities underpinning their implementation and the key differences relative to the code accompanying Nadarajah and Kotz (2006). We denote by $f_X(x; a, b, \lambda)$, $F_X(x; a, b, \lambda)$, and $F_X^{-1}(p; a, b, \lambda)$ the probability mass function, cumulative distribution function, and quantile function, respectively, of the truncated Poisson distribution targeted throughout this thesis, and by $g(x; \lambda)$, $G(x; \lambda)$, and $G^{-1}(p; \lambda)$ the corresponding functions of the non-truncated Poisson distribution, implemented in base `R` as `stats::dpois()`, `stats::ppois()`, and `stats::qpois()` respectively. Further implementation detail for every function, beyond what is summarised here, is available in the associated `roxygen2` (Wickham et al. 2026) help documentation distributed with the `truncpois` package itself.

3.1 `dtruncpois`

The probability mass function of the truncated Poisson distribution is

$$f_X(x; a, b, \lambda) = \frac{g(x; \lambda)}{G(b; \lambda) - G(a-1; \lambda)}, \qquad a \leq x \leq b,$$

implemented by `dtruncpois()` entirely on the log scale, $\log f_X(x) = \log g(x; \lambda) - \log[G(b; \lambda) - G(a-1; \lambda)]$, with the log-scale difference in the denominator computed via a numerically stable log-difference-of-

exponentials routine rather than direct subtraction. Two differences relative to Nadarajah and Kotz (2006) are worth highlighting. First, the normalising constant is computed on the log scale throughout, as just described, rather than as a direct (and potentially unstable) difference of the untruncated CDF evaluated at the bounds. Second, `dtruncpois()` allows the input `x` to lie outside $[a, b]$, returning a density of 0 (or $-\infty$ if `log = TRUE`) rather than raising an error, which is more convenient when `x` is a vector spanning values both inside and outside the truncation region.

3.2 `ptruncpois`

The cumulative distribution function is

$$F_X(x; a, b, \lambda) = \frac{G(x; \lambda) - G(a-1; \lambda)}{G(b; \lambda) - G(a-1; \lambda)}, \qquad a \le x \le b,$$

with $F_X(x) = 0$ for $x < a$ and $F_X(x) = 1$ for $x \ge b$. Relative to Nadarajah and Kotz (2006), `ptruncpois()` differs in four ways: (a) it supports `log.p = TRUE` and/or `lower.tail = FALSE`, in addition to the default lower-tail, linear-scale probability; (b) the normalising constant is computed on the log scale; (c) the input `q` may lie outside $[a, b]$, in which case the corresponding truncated probability of 0 or 1 (or $-\infty$ or 0 on the log scale) is returned rather than an error; and (d) the lower truncation bound is coerced to $a - 1$ when evaluating $G(\cdot)$, for the reason explained in Section 2.2. Point (a) deserves particular emphasis: writing the CDF explicitly in terms of $G(\cdot)$, as above, makes clear that the internal calls to $G(\cdot)$ used to compute $F_X(x)$ are always evaluated on the log scale in their lower-tail form, and the requested `lower.tail`/`log.p` combination is applied exactly once, to the final combined result, rather than being passed down to each constituent evaluation of $G(\cdot)$ individually. The code of Nadarajah and Kotz (2006), by contrast, supplies the requested `lower.tail`/`log.p` arguments directly to each constituent call to the non-truncated distribution function; because $F_X(x)$ is a ratio involving two such calls rather than a single one, this does not correctly propagate through the intermediate calculations, and can produce an incorrect result rather than merely a numerically imprecise one, as discussed in Section 2.2.

3.3 `qtruncpois`

Inverting the relationship above for a target lower-tail probability p, the value x satisfying $F_X(x) = p$ is obtained from

$$G(x; \lambda) = G(a{-}1; \lambda)\,(1{-}p) + p\,G(b; \lambda), \qquad \text{so that} \qquad F_X^{-1}(p; a, b, \lambda) = G^{-1}(G(a{-}1; \lambda)(1{-}p) + p\,G(b; \lambda);\ \lambda),$$

evaluated using the untruncated quantile function $G^{-1}(\cdot)$. This identity lets `qtruncpois()` delegate directly to `stats::qpois()` after remapping the target probability, rather than requiring a bespoke search procedure. As with `ptruncpois()`, the differences relative to Nadarajah and Kotz (2006) are: (a) support for `log.p = TRUE` and/or `lower.tail = FALSE`; (b) the log-scale computation of the remapped probability above, using a log-sum-exp routine rather than direct addition; and (c) the $a \to a - 1$

coercion. Regarding (a), `qtruncpois()` first converts whichever combination of `lower.tail` and `log.p` was supplied into a single, unified log-scale lower-tail probability, and only then applies the remapping formula above; the input `p` is therefore always assumed, by the time it reaches $G(\cdot)$ and $G^{-1}(\cdot)$, to already represent the correct tail and scale. This contrasts with the approach taken by Nadarajah and Kotz (2006), which supplies the `lower.tail`/`log.p` arguments to each constituent $G(\cdot)$ and $G^{-1}(\cdot)$ call individually, rather than accounting for them once in the overall expression.

3.4 `rtruncpois`

Three sampling algorithms are provided via the `method` argument, only one of which is provided by Nadarajah and Kotz (2006).

The **direct** method enumerates the finite support $[a, b]$ and samples via the Gumbel-max trick (Yellott 1977): for a categorical distribution with log-probabilities $\{\log f_X(k)\}_{k=a}^{b}$, adding independent standard Gumbel noise γ_k to each log-probability and taking the arg max, $\arg\max_k[\log f_X(k) + \gamma_k]$, produces a draw from the categorical distribution itself, without ever needing to compute the normalising constant explicitly. `truncpois` generates the required Gumbel noise from exponential draws, via the identity $-\log(\text{Exp}(1)) \sim \text{Gumbel}(0, 1)$. Because this method enumerates $[a, b]$ explicitly, it is, in principle, unusable when $b = \infty$; `rtruncpois()` resolves this by first shrinking the effective support to $[\max(a, q_{\text{lo}}), \min(b, q_{\text{hi}})]$, where q_{lo} and q_{hi} are extreme quantiles of the untruncated distribution, corresponding to a tail probability of $\sqrt{\varepsilon}$ (where ε is machine epsilon), obtained via $G^{-1}(\cdot)$. This is what makes `"direct"` usable even when $b = \infty$, since the excluded tail probability is smaller than is representable in floating-point arithmetic; despite the enumeration cost this introduces, `"direct"` is nonetheless the package's default method, and Section 4 compares its computational efficiency against the other two methods across a range of scenarios.

The **inversion** method applies the quantile-inversion identity of `qtruncpois()`, described above, directly to a uniform random draw: for $U \sim \text{Uniform}(0, 1)$, $F_X^{-1}(U; a, b, \lambda)$ is a draw from the truncated distribution. Rather than generating U and computing $\log(U)$ directly, `truncpois` instead draws the equivalent log-scale value via the identity $\log(\text{Uniform}(0, 1)) \sim -\text{Exp}(1)$, so that a single exponential draw supplies the log-probability input required internally, without the loss of precision that computing $\log(\texttt{runif}(n))$ directly can incur when U is close to 0. This method never enumerates the support, so it works for any a, b, and λ, including $b = \infty$.

The **bounded** method also inverts through the CDF, but avoids `qtruncpois()`'s remapping step entirely: it draws a log-scale value directly within $[\log G(a - 1; \lambda), \log G(b; \lambda)]$ — using the same $\log(\text{Uniform}(0, 1)) \sim -\text{Exp}(1)$ identity where $G(a - 1; \lambda) = 0$, and a numerically stable $\log(x + y)$ construction otherwise — and passes this value directly to the *untruncated* quantile function $G^{-1}(\cdot)$, following the general rejection-free bounded-interval inversion approach described by Zhang and Reiter (2022). Because the resulting log-scale value already lies within the correct sub-interval, no truncated normalising constant or remapping through `qtruncpois()` is required on each draw, which is what

makes `"bounded"` the fastest of the three methods whenever the truncation region retains most of the untruncated probability mass, as Section 4 demonstrates empirically.

3.5 `extruncpois` and `vartruncpois`

The mean and variance are grouped together in this subsection, since both are obtained from the same underlying Poisson recurrence relation, $k\,g(k;\lambda) = \lambda\,g(k-1;\lambda)$. Summing both sides over $k = a, \ldots, b$ gives

$$\sum_{k=a}^{b} k\,g(k;\lambda) = \lambda \sum_{k=a}^{b} g(k-1;\lambda) = \lambda\big[G(b-1;\lambda) - G(a-2;\lambda)\big],$$

so that, after normalising by $P(a \le X \le b) = G(b;\lambda) - G(a-1;\lambda)$,

$$E[X \mid a \le X \le b] = \lambda\,\frac{G(b-1;\lambda) - G(a-2;\lambda)}{G(b;\lambda) - G(a-1;\lambda)}.$$

An analogous argument using the second-order recurrence $k(k-1)\,g(k;\lambda) = \lambda^2\,g(k-2;\lambda)$ gives

$$E\big[X(X-1) \mid a \le X \le b\big] = \lambda^2\,\frac{G(b-2;\lambda) - G(a-3;\lambda)}{G(b;\lambda) - G(a-1;\lambda)},$$

from which $E[X^2] = E[X(X-1)] + E[X]$ and $\mathrm{Var}(X) = E[X^2] - E[X]^2$ follow directly. Both `extruncpois()` and `vartruncpois()` evaluate these expressions entirely via log-scale differences of `stats::ppois()` calls, so no explicit summation over the support is ever performed and the closed forms above are exact regardless of how wide $[a, b]$ is. This is a substantive departure from Nadarajah and Kotz (2006), whose code instead relies on numerical integration to obtain the mean and variance; numerical integration is a technique designed for continuous functions and is not, in general, an appropriate way to sum a discrete probability mass function. Exact summation over the support would be a valid discrete alternative, but is only computationally feasible when the support is finite (i.e. under double truncation); the recurrence-based approach used by `truncpois` avoids this restriction entirely, remaining exact and equally cheap regardless of whether b is finite or infinite. It is also worth noting that `stats::ppois()` returns 0 whenever its argument is negative, so the $a-1$, $a-2$, and $a-3$ terms appearing throughout the formulas above are automatically and safely handled without any special-casing in the implementation, even when they fall below the support.

3.6 `medtruncpois` and `modtruncpois`

The median and mode are grouped together in a separate subsection, since both are entirely new contributions relative to Nadarajah and Kotz (2006), whose code provides neither. The median is obtained as $F_X^{-1}(0.5; a, b, \lambda)$, using the same quantile-inversion identity as `qtruncpois()`; `medtruncpois()` evaluates this via the truncated CDF at the two truncation boundaries, keeping the computation on the log scale throughout, before delegating to `qtruncpois()` to obtain the corresponding quantile.

The mode of the non-truncated Poisson distribution is well known to be $\lfloor \lambda \rfloor$, with an additional tied

mode at $\lambda - 1$ whenever λ is a positive integer, since the ratio $g(k;\lambda)/g(k-1;\lambda) = \lambda/k$ equals 1 exactly at $k = \lambda$ in that case. `modtruncpois()` computes the non-truncated mode(s) via this closed-form rule and then clamps the result to $[a, b]$, returning every value $k \in [a, b]$ that remains a mode after clamping; a warning is issued whenever more than one such value is returned.

3.7 `plottruncpois`

`plottruncpois()` plots the PMF, CDF, or quantile function of a truncated Poisson distribution, selected via the `type` argument, with an optional overlay of the corresponding non-truncated Poisson distribution (`compare = TRUE`, the default) for direct visual comparison. Internally, it always calls `dtruncpois()`, `ptruncpois()`, and `qtruncpois()` with their `log`, `log.p`, and `lower.tail` arguments left at their default values of `FALSE`, `FALSE`, and `TRUE` respectively, since these are not exposed as arguments to `plottruncpois()` itself; for the CDF plot, the horizontal axis is fixed to start at 0 regardless of a, so that the effect of left-truncation remains visible. Section 4 demonstrates the PMF plot type; the CDF and quantile-function plot types are shown below.

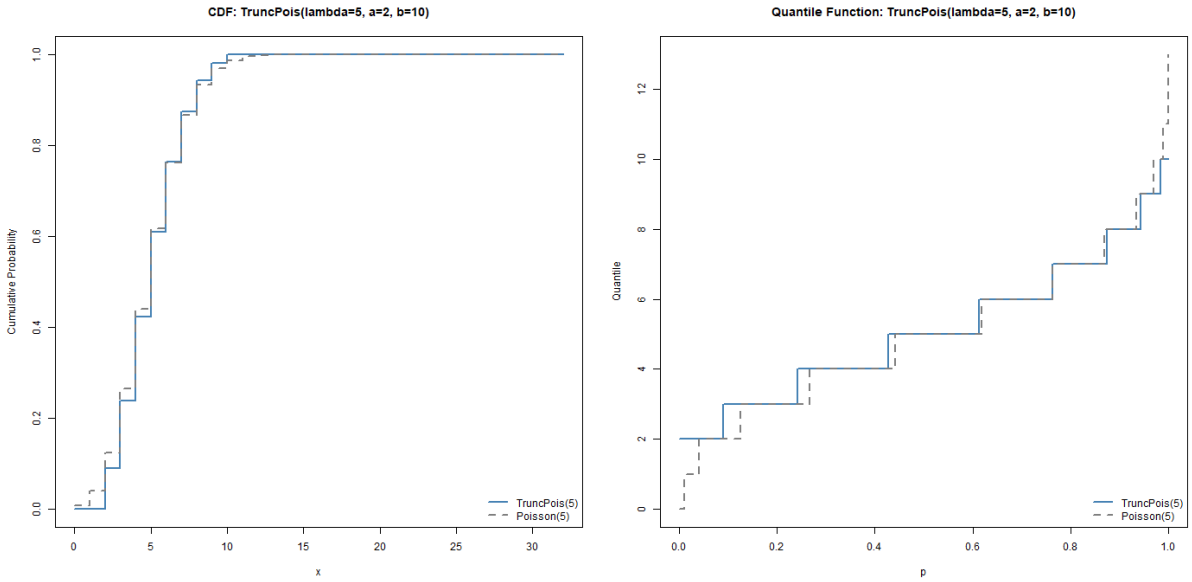

Figure 1: CDF and quantile function plots via plottruncpois(), TruncPois(λ=5, a=2, b=10) overlaid against Poisson(5)

No equivalent visualisation utility is provided by Nadarajah and Kotz (2006) or by any of the packages reviewed in Section 2.2.

3.8 `mletruncpois`

Given a sample $x_1, \ldots, x_n$ drawn independently from a truncated Poisson distribution with known bounds $[a, b]$, the log-likelihood is

$$\ell(\lambda) = \sum_{i=1}^{n} \log f_X(x_i; a, b, \lambda),$$

which, in code, is `sum(dtruncpois(x, lambda, a, b, log = TRUE))`. `mletruncpois()` maximises $\ell(\lambda)$ over λ by numerically minimising the negative log-likelihood, using one-dimensional Brent optimisation (Brent 1973) via `stats::optimize()`, seeded from a search interval centred on the sample mean. This is practical here because the non-truncated Poisson log-likelihood is well known to be strictly concave in λ, and truncation only rescales the likelihood by a λ-dependent normalising constant, so no additional multi-modality is introduced that would defeat a simple one-dimensional search. `mletruncpois()` also returns the standard error of $\hat{\lambda}$, obtained from the curvature of the negative log-likelihood at the optimum: writing H for the (scalar) Hessian of the negative log-likelihood at $\hat{\lambda}$, evaluated numerically via the `numDeriv` package (Gilbert and Varadhan 2019), the standard error is $\text{se}(\hat{\lambda}) = \sqrt{1/H}$, the usual asymptotic result for a maximum-likelihood estimator.

Every function described above validates its inputs before proceeding: `lambda` must be strictly positive, the truncation bounds must satisfy $a \geq 0$, $b > 0$, $a < b$, and both must be integer-valued (or $b = \infty$), and `rtruncpois()` additionally requires scalar (rather than vector) parameters, since it is not vectorised over λ, a, or b in the way that `dtruncpois()`, `ptruncpois()`, and `qtruncpois()` are. Invalid inputs cause an informative `stop()` with a descriptive error message; recycling of vector-valued `lambda`, `a`, or `b` arguments to a common length in `dtruncpois()`, `ptruncpois()`, and `qtruncpois()` triggers a `warning()` rather than silent recycling, to guard against accidental misuse.

3.9 Software Engineering

The package follows standard `R` (R Core Team 2026) packaging conventions to support maintainability and ease of installation. Source code is organised by functionality: `truncpois.R` implements the four core distribution functions (`dtruncpois()`, `ptruncpois()`, `qtruncpois()`, `rtruncpois()`), `moments.R` implements the closed-form mean, variance, median, and mode, `mle.R` implements `mletruncpois()`, and `plottruncpois.R` implements the visualisation utility. A further, unexported file, `zzz_utils.R`, holds a small set of private, numerically stable helper functions (log-sum-exp, log-$(1 - \exp(\cdot))$, and the log-scale normalising constant shared by the core distribution functions) that are centrally implemented and tested once, rather than duplicated across functions.

Every exported function is documented using `roxygen2` (Wickham et al. 2026), with runnable `@examples` blocks that double as informal usage demonstrations, and correctness is verified by an automated `testthat` (Wickham 2011) suite covering boundary behaviour (e.g. the PMF summing to exactly 1 over the support, and returning exactly 0 outside it), agreement with the corresponding non-truncated `stats::dpois()`/`ppois()`/`qpois()`/`rpois()` functions under default bounds, the inverse relationship between `ptruncpois()` and `qtruncpois()`, large-sample convergence of `rtruncpois()` to the closed-form moments across all three sampling methods, input validation, and a dedicated stress-test file exercising extreme parameter regimes, including $\lambda = 100{,}000$ and the narrowest possible truncation window, as demonstrated in Section 4. The package's only hard runtime dependency beyond base `R` is `numDeriv` (Gilbert and Varadhan 2019), used by `mletruncpois()` to evaluate the Hessian of the log-likelihood; `LaplacesDemon`, `microbenchmark` (Mersmann 2024), `knitr`, `rmarkdown`, and `testthat`

are listed only as optional `Suggests` dependencies, used respectively for the comparisons in Section 4, benchmarking, and vignette building. Package structure and style are further checked using `styler` (Müller et al. 2025), for consistent code formatting, `goodpractice` (Padgham et al. 2026), for general best-practice advice, and `desc` (Csárdi et al. 2023), for programmatic validation of the `DESCRIPTION` file. `truncpois` is licensed under the GNU General Public License (GPL, version ≥ 3), and development is version-controlled with `git` and hosted publicly on GitHub.

4 Results and Experiments

Six experiments demonstrate the reliability, accuracy, and computational robustness of `truncpois`. Experiment 1 visualises the PMF of a truncated Poisson distribution against its non-truncated counterpart. Experiment 2 demonstrates the closed-form convergence of the truncated mean and variance to their non-truncated values as the truncation window widens; since `LaplacesDemon` has no closed-form moments for the truncated Poisson distribution at all, no direct comparison is shown there. Experiment 3 compares the computational efficiency of the package's three sampling algorithms, and of `LaplacesDemon`, across five diverse truncation scenarios. Experiment 4 demonstrates numerical stability at extreme parameter values and narrow truncation windows, and compares directly against `LaplacesDemon`, exposing two concrete defects in the latter discussed in Section 2.2. Experiment 5 simulates data from a zero-truncated Poisson distribution and validates the package's sampling and moment functions against it. Experiment 6 recovers a known rate parameter from simulated data via `mletruncpois()`.

4.1 Experiment 1: Visualization of Probability Mass Function

This experiment shows how the PMF of the non-truncated Poisson distribution differs from the PMF of a right-truncated Poisson distribution, generated directly via the package's own `plottruncpois()` visualization utility; the truncation bounds are shown directly in the figure rather than in a plot title, consistent with the captioning convention used throughout this thesis.

Conclusion: Truncation concentrates the probability mass onto the interval $[0, 10]$, with the largest visible increase in probability occurring near the upper bound $b = 10$, where mass that would otherwise fall in the excluded right tail is redistributed.

4.2 Experiment 2: Moment Convergence

As the truncation boundaries become larger, the moments of the truncated distribution tend towards the corresponding moments of the untruncated distribution. The following experiment proves the convergence of mean and variance.

Conclusion: Both the mean and the variance of the right-truncated distribution converge to $\lambda = 5$ as the upper bound b grows, recovering the untruncated Poisson moments once b is large enough that

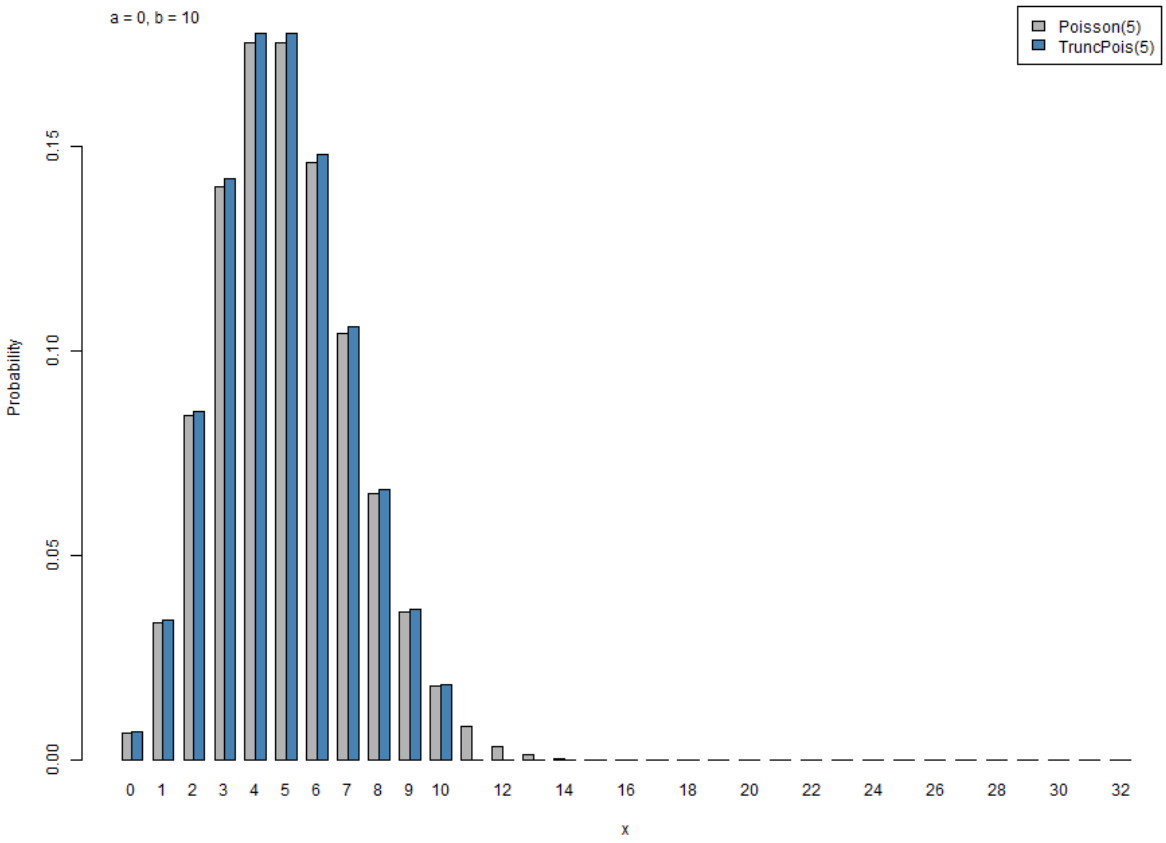

Figure 2: Visualization of PMF: non-truncated vs right-truncated Poisson distribution (λ=5, b=10)

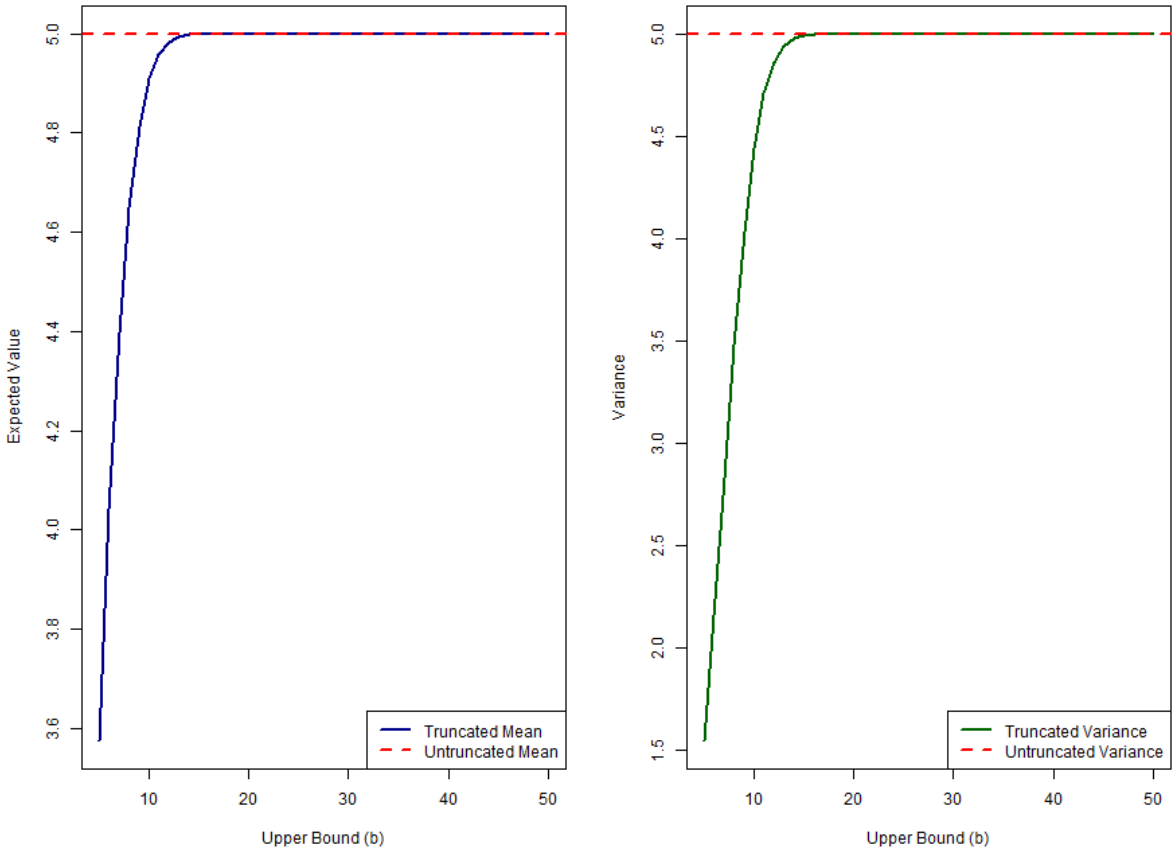

Figure 3: Mean and variance convergence of a truncated Poisson(λ=5) distribution

truncation removes a negligible share of the probability mass.

Table 1 reports the exact closed-form mean and variance at selected upper bounds, computed directly via `extruncpois()` and `vartruncpois()`. Both moments are visibly below their non-truncated values of 5 for small b, since restricting the support to $[0, b]$ removes the right tail of the distribution and pulls the conditional mean and variance downward; by $b = 20$ both moments have converged to the non-truncated values to within rounding precision, since the non-truncated Poisson(5) places negligible probability mass above 20.

More generally, truncation always reduces the variance of the distribution, regardless of which side is truncated, since restricting the support necessarily removes probability mass from at least one tail. The effect on the mean, however, depends on the direction of truncation: right-truncation, as demonstrated here, pulls the conditional mean downward, since it removes the upper tail; left-truncation has the opposite effect, pulling the conditional mean upward, since it instead removes the lower tail (including, in the zero-truncated case, the point mass at $X = 0$).

Table 1: Exact mean and variance of the right-truncated Poisson($\lambda = 5$, $a = 0$, b) at selected upper bounds, computed via `extruncpois()` and `vartruncpois()`.

Upper bound b	5	10	20	30	50
Mean	3.576	4.908	5.000	5.000	5.000
Variance	1.547	4.440	5.000	5.000	5.000

No direct comparison to `LaplacesDemon` is shown for this experiment, since it provides no closed-form mean or variance for the truncated Poisson distribution at all; the only way to approximate these moments via `LaplacesDemon` is by simulating a large sample from `rtrunc()` and computing the sample mean and variance, which is strictly less precise, and slower, than the exact values reported here.

4.3 Experiment 3: Sampling Method Efficiency

The `rtruncpois()` function provides three sampling algorithms optimized for different parameter configurations. This experiment compares their computational efficiency, and that of `LaplacesDemon::rtrunc()`, across five diverse truncation scenarios using `microbenchmark::microbenchmark()`: a left-truncated (zero-truncated) case with unbounded support, a wide right-truncated case, a narrow doubly-truncated case with λ centered in the window, a narrower doubly-truncated case with λ near the edge of the window, and a doubly-truncated case with λ positioned well outside the truncation window entirely.

Table 2 reports the median wall-clock time (milliseconds) to draw 10,000 samples under each method and scenario, taken over 20 `microbenchmark` repetitions per cell. `LaplacesDemon::rtrunc()` was called with a manually corrected to $a - 1$ throughout, so that the timing comparison is made on equal, correct terms.

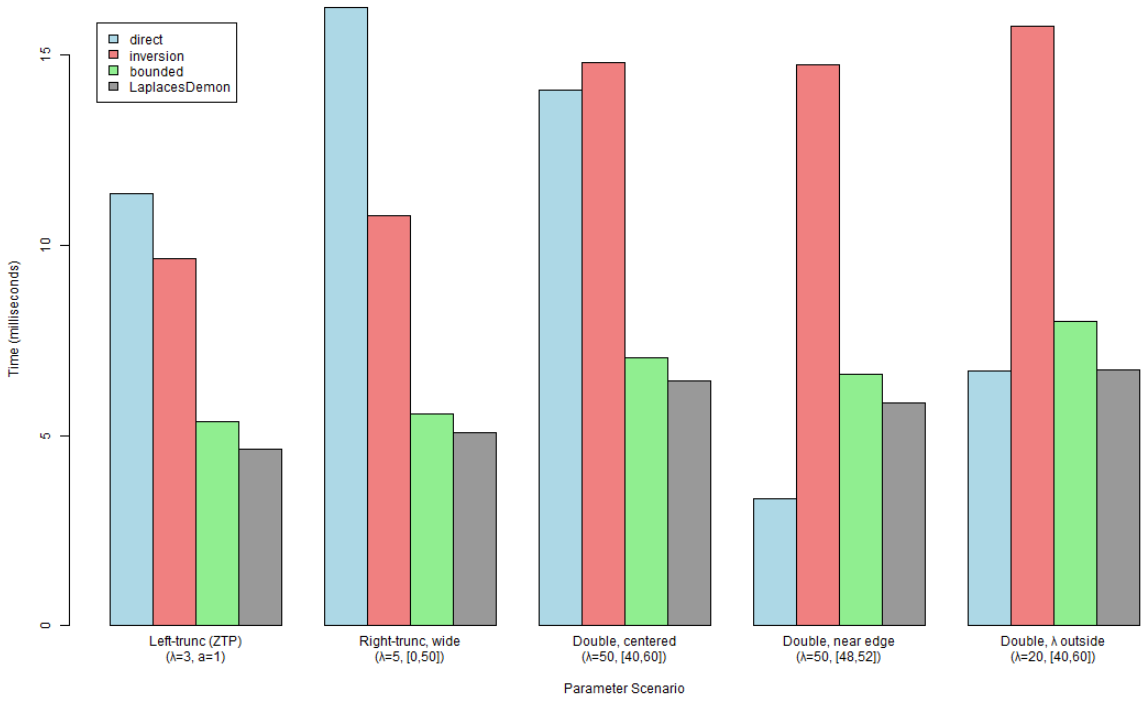

Figure 4: Sampling method efficiency benchmark

Table 2: Median time (ms) to draw 10,000 samples per method and scenario, over 20 `microbenchmark::microbenchmark()` repetitions.

Method	ZTP (λ=3, a=1)	Right, wide (λ=5, [0,50])	Double, centered (λ=50, [40,60])	Double, near edge (λ=50, [48,52])	λ outside (λ=20, [40,60])
Direct	11.89	15.35	14.33	3.66	7.29
Inversion	9.97	10.69	14.79	15.69	16.77
Bounded	5.51	5.58	7.07	6.99	8.43
LaplacesDemon	4.71	4.90	6.64	6.32	7.39

Interpretation: `LaplacesDemon`'s `rtrunc()` is the fastest method in every scenario tested here, since it performs the same style of direct CDF-inversion as `truncpois`'s `"bounded"` method but with less input-validation overhead; raw sampling speed was never the primary claim motivating `truncpois`'s design, and this comparison should be read alongside two important caveats. First, `rtrunc()` only produced these timings once a was manually corrected to $a - 1$ for every scenario in this table; passed naively, as a user unaware of the distinction demonstrated in Section 2.2 would do, it silently returns samples from the wrong distribution rather than raising an error, a failure mode `truncpois` does not have. Second, `LaplacesDemon` provides no way to obtain closed-form moments, medians, modes, or maximum-likelihood estimates from the samples it generates, all of which `truncpois` provides directly. Among `truncpois`'s own three methods, `"bounded"` is consistently the fastest, since, like `LaplacesDemon`, it performs a

single vectorized CDF-inversion pass through the non-truncated Poisson quantile function without ever enumerating the support or remapping through `qtruncpois()`. The `"direct"` method's cost is more variable, since it must enumerate the finite support and generate an independent Gumbel variate per sample per support point: it is the fastest of the three `truncpois` methods precisely when the support is narrowest (the *near edge* scenario, with only five support points), and the slowest of the three when the support is comparatively wide (the *right-truncated, wide* scenario). Despite this variability, `"direct"` remains the package's default method: because it samples in direct proportion to the values returned by `dtruncpois()` itself, it is correct by construction whenever the PMF is correct, without depending on `stats::qpois()`'s numerical search correctly inverting an extreme log-probability, as both `"inversion"` and `"bounded"` do. The `"inversion"` method is consistently the slowest of the three `truncpois` methods tested here, incurring the overhead of `qtruncpois()`'s probability-remapping step on every draw, on top of the same reliance on `stats::qpois()` that `"bounded"` has; it is retained chiefly as a simple, transparent implementation built directly on `qtruncpois()`.

4.4 Experiment 4: Numerical Stability Demonstration

A key advantage of `truncpois` is its ability to handle extreme parameter values without numerical underflow. This experiment demonstrates log-scale computations across a wide range of λ values, extended to include the stress-test extremes exercised by the package's own test suite: $\lambda = 100{,}000$ with a narrow truncation window, and the narrowest possible truncation window (b = a + 1).

Interpretation: The log-PMF remains computable for all tested λ values (10 to 100,000) with narrow truncation windows, and the narrowest possible truncation window remains numerically well-behaved. Traditional naive implementations would produce zero or underflow errors in these regimes; `truncpois` handles them seamlessly via log-space arithmetic.

Table 3: Log-PMF at $\lfloor\lambda\rfloor$ for each tested parameter regime, computed via `dtruncpois(..., log = TRUE)`. These particular evaluation points sit near the mode of each regime and so are not themselves at risk of underflow; the numerical-stability benefit of computing entirely on the log scale is most apparent instead when evaluating the normalizing constant $G(b) - G(a-1)$ for windows placed further into the tails of very large-λ distributions, where a naive linear-scale subtraction of two near-identical cumulative probabilities is vulnerable to catastrophic cancellation, a risk that log-scale differencing (via the `.log_diff()` helper in `zzz_utils.R`) avoids entirely, regardless of where the window is placed.

λ	a	b	Log-PMF at $\lfloor\lambda\rfloor$
10	0	20	−2.077
50	40	60	−2.730
100	90	110	−2.875
200	190	210	−2.956
500	490	510	−3.008
100,000	99,500	100,500	−6.555
5 *(narrowest window)*	3	4	−0.811

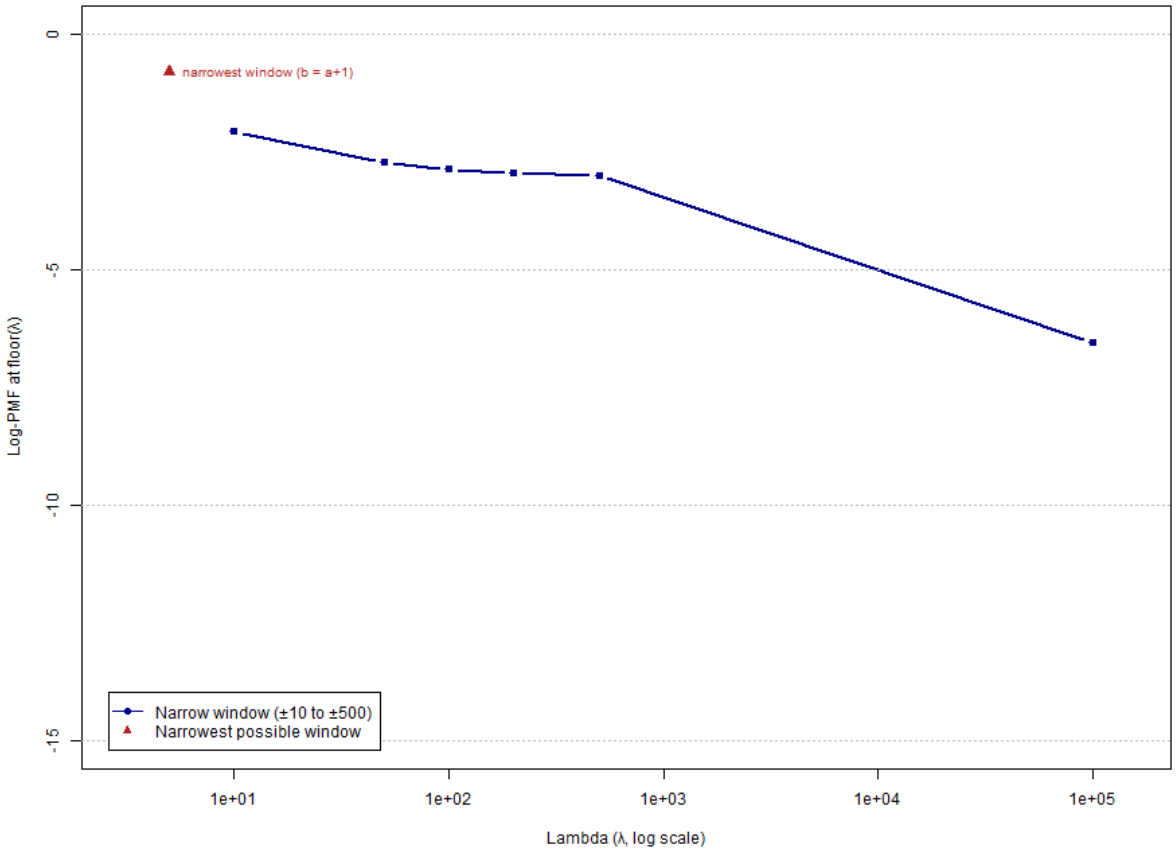

Figure 5: Numerical stability across extreme parameters

This experiment also provides an opportunity to demonstrate, concretely, several of the defects in `LaplacesDemon` discussed in Section 2.2, rather than merely asserting them, and to extend the numerical-stability comparison beyond `dtruncpois()` alone to `ptruncpois()` and `qtruncpois()` as well. Table 3b repeats the same six parameter regimes above, adding the log-PMF that `LaplacesDemon::dtrunc()` returns when a is passed directly, without the $a \to a - 1$ correction that a user unfamiliar with the discrete-truncation issue would naturally omit.

Table 4: Log-PMF at $\lfloor \lambda \rfloor$ from `truncpois` versus `LaplacesDemon` called with the uncorrected lower bound a.

			`truncpois`	`LaplacesDemon`
λ	a	b	log-PMF	log-PMF (naive a)
10	0	20	−2.077	−2.077
50	40	60	−2.730	−2.704
100	90	110	−2.875	−2.839
200	190	210	−2.956	−2.914
500	490	510	−3.008	−2.962
100,000	99,500	100,500	−6.555	−6.555

The discrepancy is smallest in the first and last rows, where a is either 0 or so far into the tail that $G(a; \lambda)$ is already negligible, and largest in the middle rows, where a carries a non-trivial share of the untruncated distribution's probability mass; in every such case, `LaplacesDemon` silently returns a subtly incorrect density rather than raising an error. A second, unrelated defect is demonstrated in Table 3c: `LaplacesDemon::ptrunc()` accepts a `lower.tail` argument without error, but does not actually use it.

Table 5: `ptruncpois()` versus `LaplacesDemon::ptrunc()` for $P(X \leq 7 \mid 2 \leq X \leq 10, \lambda = 5)$, with `lower.tail = TRUE` and `lower.tail = FALSE`.

`lower.tail`	`ptruncpois()`	`LaplacesDemon::ptrunc()`
`TRUE`	0.873	0.873
`FALSE`	0.127	0.873

Passing `lower.tail = FALSE` returns exactly the same value as the (lower-tail) default, confirming that the argument is silently ignored rather than merely undocumented; this is a considerably more dangerous failure mode than requiring a manual `1 - p` transformation, since it gives no indication to the user that anything is wrong. The same defect affects `LaplacesDemon::qtrunc()`, and more severely: Table 3d shows that it too silently ignores `lower.tail`, and, unlike `ptrunc()`, it does not accept `log.p` as an argument at all, raising the error `"p must be in [0,1]"` if a log-probability is supplied, rather than either computing the correct log-scale quantile or failing with an informative message about the unsupported argument.

Table 6: `qtruncpois()` versus `LaplacesDemon::qtrunc()` for the 0.3 quantile of $X \mid 2 \leq X \leq 10, \lambda = 5$, with `lower.tail = TRUE` and `lower.tail = FALSE`.

lower.tail	qtruncpois()	LaplacesDemon::qtrunc()
TRUE	4	4
FALSE	6	4

4.5 Experiment 5: Zero-Truncated Poisson Simulation

Zero-truncated Poisson (ZTP) distributions are widely used in count models where zero outcomes are structurally impossible to observe. For example, when hospitals record the number of visits made by individual patients, the resulting counts are zero-truncated, since a patient who never visited is not present in the hospital's records at all; similarly, the number of times an individual visits a tourist site is often recorded only on-site, so a zero-visit count is never observed either. This experiment simulates data from a ZTP distribution and demonstrates sampling, moment recovery, and model fit via visualization.

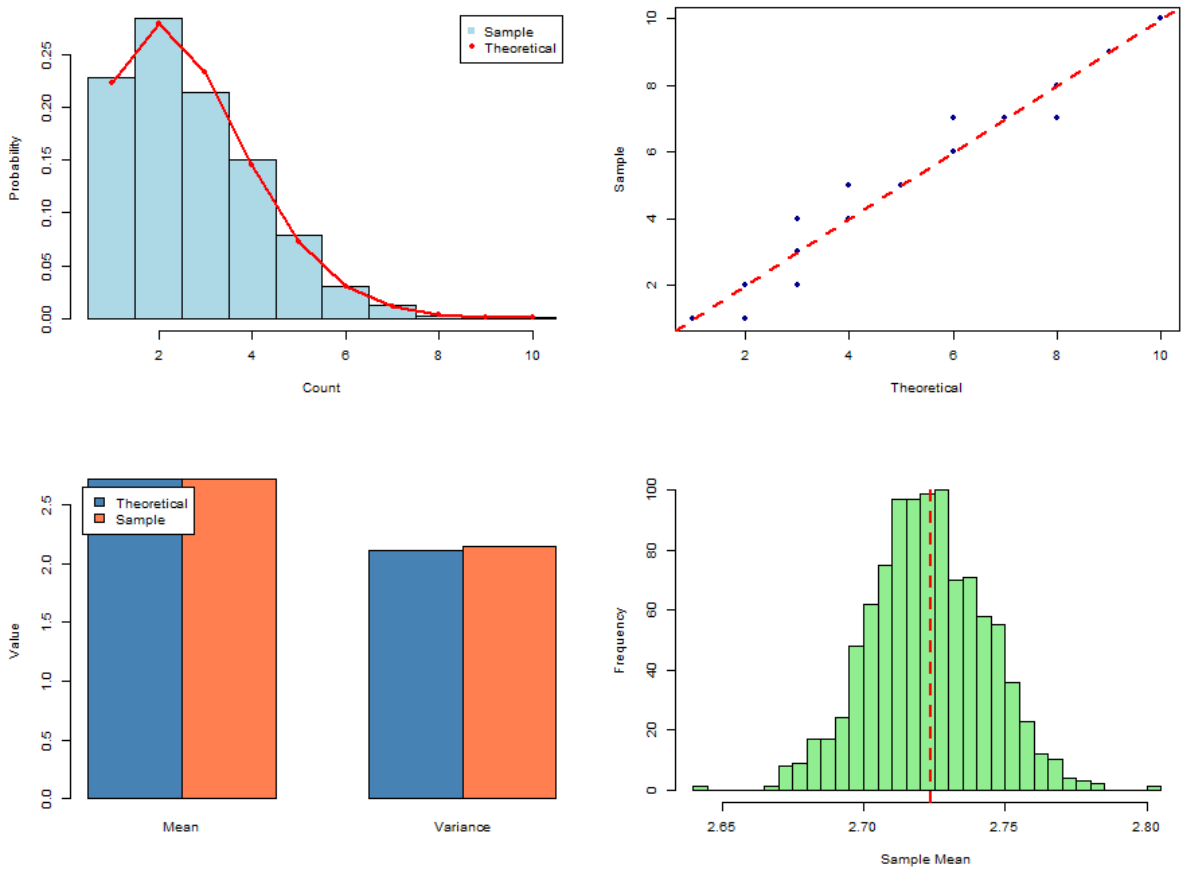

Figure 6: Zero-Truncated Poisson simulation: sampling, moment recovery, and bootstrap inference

Interpretation: The four panels show (clockwise from top-left): the histogram of 5,000 simulated ZTP samples with theoretical PMF overlay, a Q-Q plot validating distributional fit, a comparison of theoretical

vs. empirical moments, and a bootstrap distribution of the sample mean. All demonstrate good agreement between theory and simulation.

This scenario is representative of the applied use case motivating the package: a count variable (e.g. number of insurance claims, or number of hospital visits) that is only observed conditional on being at least 1. Fitting a standard, non-truncated Poisson model to such data without accounting for the missing zero category would systematically overestimate λ, since the observed sample mean of 2.72 (bootstrap distribution, right panel) is itself already a biased estimator of the *non-truncated* rate parameter; it instead estimates the conditional mean $E[X \mid X \geq 1]$, which is always strictly greater than λ itself. It is important to note that `extruncpois()` cannot be used to correct for this: it computes the theoretical mean of the truncated distribution *given* a known λ, and is not an estimator of λ from data. Recovering λ from an observed, zero-truncated sample instead requires `mletruncpois()`, demonstrated next in Experiment 6, since the sample mean of a zero-truncated sample is not, in general, equal to the maximum-likelihood estimate of λ.

4.6 Experiment 6: MLE Parameter Recovery

This experiment demonstrates `mletruncpois()`, which estimates λ from a sample of observed counts. A sample is simulated from a truncated Poisson distribution with a known λ, the parameter is re-estimated from the sample alone via maximum likelihood, and the fitted distribution is compared against both the simulated data and the true generating distribution.

Interpretation: The maximum-likelihood estimate of λ recovered from the sample closely matches the true generating value, and the theoretical PMF evaluated at the fitted λ closely tracks the empirical sample proportions. This confirms that `rtruncpois()`, `mletruncpois()`, and the closed-form moment functions are mutually consistent.

The fitted value was $\hat{\lambda} = 6.037$ against a true generating $\lambda = 6$, from $n = 5{,}000$ samples (log-likelihood at $\hat{\lambda}$: $-11{,}184.6$). Table 4 compares the sample moments directly against the theoretical moments evaluated at $\hat{\lambda}$ via `extruncpois()`, `vartruncpois()`, `medtruncpois()`, and `modtruncpois()`.

Table 7: Sample moments of the simulated data versus theoretical moments evaluated at the fitted $\hat{\lambda} = 6.037$.

Statistic	Sample	Theoretical (at $\hat{\lambda}$)
Mean	6.120	6.120
Variance	5.497	5.615
Median	6	6
Mode	6	6

The close agreement across all four moments, particularly the mean, which matches to three decimal places since $\hat{\lambda}$ was itself chosen to maximize the likelihood of the observed sample, provides an internal consistency check spanning three separate parts of the package: random generation (`rtruncpois()`),

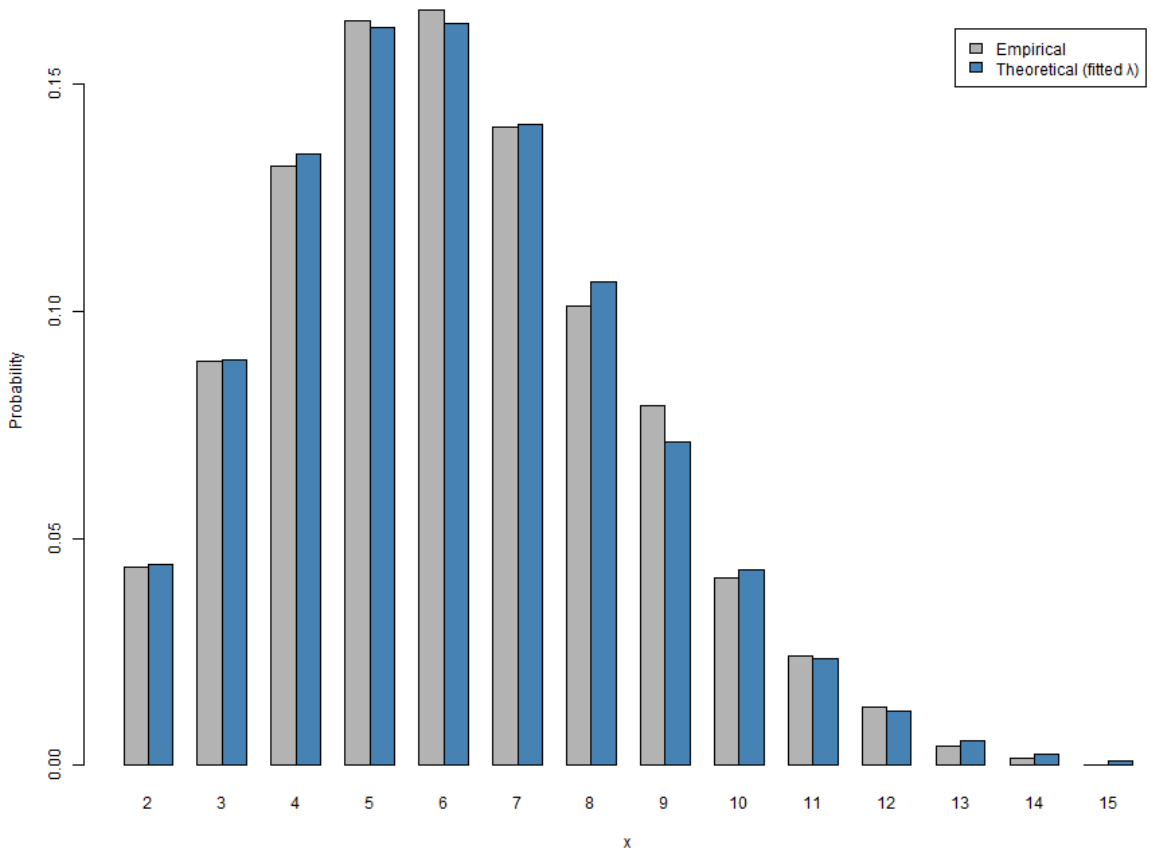

Figure 7: MLE parameter recovery: fitted vs true truncated Poisson

estimation (`mletruncpois()`), and the closed-form moment functions, all evaluated independently of one another yet arriving at compatible answers.

---

5 Discussion

5.1 Limitations

While `truncpois` addresses the specific gaps identified in Section 2.2, several limitations remain and are worth stating explicitly.

- **Single-distribution scope.** The package currently supports only the truncated Poisson distribution. Many of the same numerical-stability and closed-form-moment arguments apply equally to other truncated discrete distributions (negative binomial, binomial, geometric), but these are not yet implemented; a user needing truncated negative-binomial support, for instance, would need to look elsewhere (or to the generic, non-closed-form `truncdist` package).
- **No covariate-dependent estimation.** `mletruncpois()` estimates a single scalar λ from a homogeneous sample with fixed, known truncation bounds. It does not support regression-style modelling where λ depends on covariates, unlike the dedicated regression frameworks `countreg::zerotrunc()` and `gamlss.tr`. Extending `truncpois` to a regression setting would require substantially different machinery (e.g. IRLS or full likelihood optimization over a coefficient vector) beyond the one-dimensional `stats::optimize()` approach used here.
- **Optimization-based (not closed-form) MLE.** Unlike the moment functions, `mletruncpois()` relies on numerical maximization of the log-likelihood rather than a closed-form estimating equation. Although the search interval is seeded near the sample mean and the truncated Poisson likelihood is well-behaved in practice (Section 4.6), this has not been formally proven to guarantee global convergence in all conceivable edge cases, e.g. very small samples with heavily truncated support.
- **No compiled backend.** All computation is implemented in base R; a compiled (e.g. Rcpp) backend was considered out of scope for this thesis but could offer speed benefits for very large-scale simulation workloads, particularly for the direct sampling method, whose cost scales with support width (Section 4.3).
- **Not yet on CRAN.** The package is currently distributed via GitHub only, has not undergone `R CMD check --as-cran` submission review, and has not been exposed to the broader scrutiny (and edge-case bug reports) that CRAN release and public usage would bring.

5.2 Conclusions

The experiments in Section 4 collectively demonstrate that `truncpois` behaves correctly and stably across the parameter regimes it was designed for: PMF and CDF values agree with the standard Poisson

distribution functions under default (untruncated) bounds, closed-form moments converge to their untruncated counterparts as the truncation window widens, all three sampling algorithms converge to the correct closed-form moments at large sample sizes, log-scale computations remain well-behaved even at $\lambda = 100{,}000$ and under the narrowest possible truncation window, and the maximum-likelihood estimator recovers the true generating parameter from simulated data. Taken together with the literature comparison in Section 2.2, these results support the central claim of this thesis: that `truncpois` fills a real gap in R's ecosystem for exact, numerically stable, Poisson-specific truncated-distribution computation.

---

6 Conclusion

6.1 Summary

The `truncpois` R package offers a complete and numerically stable implementation of truncated Poisson distributions. Through the use of closed-form moment calculations and adaptive sampling techniques, it addresses critical shortcomings found in previous implementations.

6.2 Key Contributions

1. **Completeness** – Full distributional inference via d, p, q, r, and moment functions
2. **Precision** – Closed-form moments eliminate simulation error
3. **Stability** – Log-scale arithmetic handles extreme parameter regimes
4. **Efficiency** – Multiple sampling algorithms optimized for different scenarios
5. **Reproducibility** – Deterministic, seed-independent moment calculations
6. **Parameter Estimation** – Maximum likelihood estimation of λ directly from observed count data, via `mletruncpois()`

6.3 Future Directions

Potential enhancements include: - Extension to other truncated discrete distributions (negative binomial, binomial) - Graphical diagnostics and goodness-of-fit tests - Computational optimization via C/C++ implementation for large-scale simulations

---

References

Brent, R. P. (1973), *Algorithms for minimization without derivatives*, Englewood Cliffs, N.J.: Prentice-Hall.

Csárdi, G., Müller, K., and Hester, J. (2023), *Desc: Manipulate DESCRIPTION files*. https://doi.org/10.32614/CRAN.package.desc.

Gilbert, P., and Varadhan, R. (2019), *numDeriv: Accurate numerical derivatives*. https://doi.org/10.32614/CRAN.package.numDeriv.

Mersmann, O. (2024), *Microbenchmark: Accurate timing functions*. https://doi.org/10.32614/CRAN.package.microbenchmark.

Müller, K., Walthert, L., and Patil, I. (2025), *Styler: Non-invasive pretty printing of r code*. https://doi.org/10.32614/CRAN.package.styler.

Nadarajah, S., and Kotz, S. (2006), "R programs for computing truncated distributions," *Journal of Statistical Software*, 16, 1–8. https://doi.org/10.18637/jss.v016.c02.

Novomestky, F., and Nadarajah, S. (2016), *Truncdist: Truncated random variables*.

Padgham, M., Marks, K., de Bortoli, D., Csardi, G., Frick, H., Jones, O., Alexander, H., and Mowinckel, A. M. (2026), *Goodpractice: Advice on r package building*.

R Core Team (2026), *R: A language and environment for statistical computing*, Vienna, Austria: R Foundation for Statistical Computing.

Stasinopoulos, M., and Rigby, B. (2024), *Gamlss.tr: Generating and fitting truncated "gamlss.family" distributions*.

Statisticat, LLC (2026), *LaplacesDemon: Complete environment for bayesian inference*.

Wickham, H. (2011), "Testthat: Get started with testing," *The R Journal*, 3, 5–10.

Wickham, H., Danenberg, P., Csárdi, G., and Eugster, M. (2026), *roxygen2: In-line documentation for r*. https://doi.org/10.32614/CRAN.package.roxygen2.

Yee, T. (2025a), *VGAM: Vector generalized linear and additive models*.

Yee, T. (2025b), *VGAMdata: Data supporting the 'VGAM' package*.

Yellott, J. I., Jr. (1977), "The relationship between Luce's choice axiom, Thurstone's theory of comparative judgment, and the double exponential distribution," *Journal of Mathematical Psychology*, 15, 109–144.

Zeileis, A., and Kleiber, C. (2024), *Countreg: Count data regression*.

Zhang, X., and Reiter, J. P. (2022), "A generator for generalized inverse Gaussian distributions."